torch.linalg.eigh#

https://pytorch.org/docs/stable/generated/torch.linalg.eigh.html

Description:

Computes the eigenvalue decomposition of a complex Hermitian or real symmetric matrix. Letting K\mathbb{K}K be R\mathbb{R}R or C\mathbb{C}C, the eigenvalue decomposition of a complex Hermitian or real symmetric matrix A∈Kn×nA \in \mathbb{K}^{n \times n}A∈Kn×n is defined as where QHQ^{\text{H}}QH is the conjugate transpose when QQQ is complex, and the transpose when QQQ is real-valued. QQQ is orthogonal in the real case and unitary in the complex case. Supports input of float, double, cfloat and cdouble dtypes. Also supports batches of matrices, and if A is a batch of matrices then the output has the same batch dimensions. A is assumed to be Hermitian (resp. symmetric), but this is not checked internally, instead:

Function Signature

```
torch.linalg.eigh(A, UPLO='L', *, out=None)#
Parameters
Keyword Arguments
Returns
Examples::
```

Parameters

Keyword

out (tuple, optional) – output tuple of two tensors. Ignored if None. Default: None.

Returns:

A named tuple (eigenvalues, eigenvectors) which corresponds to Λ\\LambdaΛ and QQQ above. eigenvalues will always be real-valued, even when A is complex. It will also be ordered in ascending order. eigenvectors will have the same dtype as A and will contain the eigenvectors as its columns.

Code Examples

```
>>> A = torch.randn(2, 2, dtype=torch.complex128)
>>> A = A + A.T.conj()  # creates a Hermitian matrix
>>> A
tensor([[2.9228+0.0000j, 0.2029-0.0862j],
        [0.2029+0.0862j, 0.3464+0.0000j]],
dtype=torch.complex128)
>>> L, Q = torch.linalg.eigh(A)
>>> L
tensor([0.3277, 2.9415], dtype=torch.float64)
>>> Q
tensor([[-0.0846+-0.0000j, -0.9964+0.0000j],
        [ 0.9170+0.3898j, -0.0779-0.0331j]],
dtype=torch.complex128)
>>> torch.dist(Q @ torch.diag(L.cdouble()) @ Q.T.conj(), A)
tensor(6.1062e-16, dtype=torch.float64)
```

```
>>> A = torch.randn(3, 2, 2, dtype=torch.float64)
>>> A = A + A.mT  # creates a batch of symmetric matrices
>>> L, Q = torch.linalg.eigh(A)
>>> torch.dist(Q @ torch.diag_embed(L) @ Q.mH, A)
tensor(1.5423e-15, dtype=torch.float64)
```

Notes & Warnings

NOTE Note When inputs are on a CUDA device, this function synchronizes that device with the CPU.

NOTE Note The eigenvalues of real symmetric or complex Hermitian matrices are always real.

⚠ WARNING Warning The eigenvectors of a symmetric matrix are not unique, nor are they continuous with respect to A. Due to this lack of uniqueness, different hardware and software may compute different eigenvectors. This non-uniqueness is caused by the fact that multiplying an eigenvector by -1 in the real case or by eiϕ,ϕ∈Re^{i \phi}, \phi \in \mathbb{R}eiϕ,ϕ∈R in the complex case produces another set of valid eigenvectors of the matrix. For this reason, the loss function shall not depend on the phase of the eigenvectors, as this quantity is not well-defined. This is checked for complex inputs when computing the gradients of this function. As such, when inputs are complex and are on a CUDA device, the computation of the gradients of this function synchronizes that device with the CPU.

⚠ WARNING Warning Gradients computed using the eigenvectors tensor will only be finite when A has distinct eigenvalues. Furthermore, if the distance between any two eigenvalues is close to zero, the gradient will be numerically unstable, as it depends on the eigenvalues λi\lambda_iλi through the computation of 1mini≠jλi−λj\frac{1}{\min_{i \neq j} \lambda_i - \lambda_j}mini=jλi−λj1.

⚠ WARNING Warning User may see pytorch crashes if running eigh on CUDA devices with CUDA versions before 12.1 update 1 with large ill-conditioned matrices as inputs. Refer to Linear Algebra Numerical Stability for more details. If this is the case, user may (1) tune their matrix inputs to be less ill-conditioned, or (2) use torch.backends.cuda.preferred_linalg_library() to try other supported backends.

NOTE See also torch.linalg.eigvalsh() computes only the eigenvalues of a Hermitian matrix. Unlike torch.linalg.eigh(), the gradients of eigvalsh() are always numerically stable. torch.linalg.cholesky() for a different decomposition of a Hermitian matrix. The Cholesky decomposition gives less information about the matrix but is much faster to compute than the eigenvalue decomposition. torch.linalg.eig() for a (slower) function that computes the eigenvalue decomposition of a not necessarily Hermitian square matrix. torch.linalg.svd() for a (slower) function that computes the more general SVD decomposition of matrices of any shape. torch.linalg.qr() for another (much faster) decomposition that works on general matrices.

Detailed Documentation

Documentation

Computes the eigenvalue decomposition of a complex Hermitian or real symmetric matrix.

Letting K\mathbb{K}K be R\mathbb{R}R or C\mathbb{C}C, the eigenvalue

decomposition of a complex Hermitian or real symmetric matrix A∈Kn×nA \in \mathbb{K}^{n \times n}A∈Kn×n is defined as

where QHQ^{\text{H}}QH is the conjugate transpose when QQQ is complex, and the transpose when QQQ is real-valued. QQQ is orthogonal in the real case and unitary in the complex case.

Supports input of float, double, cfloat and cdouble dtypes. Also supports batches of matrices, and if A is a batch of matrices then the output has the same batch dimensions.

A is assumed to be Hermitian (resp. symmetric), but this is not checked internally, instead:

If UPLO= 'L' (default), only the lower triangular part of the matrix is used in the computation.

If UPLO= 'U', only the upper triangular part of the matrix is used.

The eigenvalues are returned in ascending order.

When inputs are on a CUDA device, this function synchronizes that device with the CPU.

The eigenvalues of real symmetric or complex Hermitian matrices are always real.

The eigenvectors of a symmetric matrix are not unique, nor are they continuous with respect to A. Due to this lack of uniqueness, different hardware and software may compute different eigenvectors.

This non-uniqueness is caused by the fact that multiplying an eigenvector by -1 in the real case or by eiϕ,ϕ∈Re^{i \phi}, \phi \in \mathbb{R}eiϕ,ϕ∈R in the complex case

produces another set of valid eigenvectors of the matrix. For this reason, the loss function shall not depend on the phase of the eigenvectors, as this quantity is not well-defined. This is checked for complex inputs when computing the gradients of this function. As such, when inputs are complex and are on a CUDA device, the computation of the gradients of this function synchronizes that device with the CPU.

Gradients computed using the eigenvectors tensor will only be finite when A has distinct eigenvalues. Furthermore, if the distance between any two eigenvalues is close to zero, the gradient will be numerically unstable, as it depends on the eigenvalues λi\lambda_iλi through the computation of 1mini≠jλi−λj\frac{1}{\min_{i \neq j} \lambda_i - \lambda_j}mini=jλi−λj1.

User may see pytorch crashes if running eigh on CUDA devices with CUDA versions before 12.1 update 1 with large ill-conditioned matrices as inputs. Refer to Linear Algebra Numerical Stability for more details. If this is the case, user may (1) tune their matrix inputs to be less ill-conditioned, or (2) use torch.backends.cuda.preferred_linalg_library() to try other supported backends.

torch.linalg.eigvalsh() computes only the eigenvalues of a Hermitian matrix. Unlike torch.linalg.eigh(), the gradients of eigvalsh() are always numerically stable.

torch.linalg.cholesky() for a different decomposition of a Hermitian matrix. The Cholesky decomposition gives less information about the matrix but is much faster to compute than the eigenvalue decomposition.

torch.linalg.eig() for a (slower) function that computes the eigenvalue decomposition of a not necessarily Hermitian square matrix.

torch.linalg.svd() for a (slower) function that computes the more general SVD decomposition of matrices of any shape.

torch.linalg.qr() for another (much faster) decomposition that works on general matrices.

A (Tensor) – tensor of shape (*, n, n) where * is zero or more batch dimensions consisting of symmetric or Hermitian matrices.

UPLO ('L', 'U', optional) – controls whether to use the upper or lower triangular part of A in the computations. Default: ‘L’.

out (tuple, optional) – output tuple of two tensors. Ignored if None. Default: None.

A named tuple (eigenvalues, eigenvectors) which corresponds to Λ\\LambdaΛ and QQQ above.

eigenvalues will always be real-valued, even when A is complex. It will also be ordered in ascending order.

eigenvectors will have the same dtype as A and will contain the eigenvectors as its columns.
